Tu es administrateur de bases de données. Voici le script de création d'une base MySQL 8 pour une plateforme de location de logements entre particuliers (type Airbnb), puis ses contraintes de validation. Génère un jeu de données de test pour remplir toutes les tables.

TABLES

```
CREATE DATABASE IF NOT EXISTS plateforme_location;

USE plateforme_location;


CREATE TABLE zone_geo (
  id_zone INT NOT NULL,
  nom_zone VARCHAR(80) NOT NULL,
  id_zone_parent INT,
  PRIMARY KEY (id_zone),
  FOREIGN KEY (id_zone_parent) REFERENCES zone_geo(id_zone)
    ON UPDATE CASCADE
    ON DELETE SET NULL
);



CREATE TABLE utilisateur (
  email_utilisateur   VARCHAR(100) NOT NULL,
  mot_de_passe       VARCHAR(255) NOT NULL,
  nom_utilisateur    VARCHAR(50)  NOT NULL,
  prenom_utilisateur  VARCHAR(50)  NOT NULL,
  date_naissance     DATE        NOT NULL,
  date_inscription   DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  statut_compte      VARCHAR(10)  NOT NULL DEFAULT 'actif',
  iban               VARCHAR(34),
```

```
  CONSTRAINT utilisateur PRIMARY KEY (email_utilisateur)
);


CREATE TABLE logement (
  id_logement     INT  NOT NULL PRIMARY KEY,
  titre_logement   VARCHAR(100)  NOT NULL,
  type_logement    VARCHAR(30)   NOT NULL,
  capacite_max    INT     NOT NULL,
  prix_nuit       DECIMAL(7,2)  NOT NULL,
  adresse_rue     VARCHAR(150)  NOT NULL,
  code_postal     VARCHAR(10)   NOT NULL,
  latitude        DECIMAL(9,6)  NOT NULL,
  longitude       DECIMAL(10,6) NOT NULL,
  email_hote      VARCHAR(100)  NOT NULL,
  id_zone         INT         NOT NULL,
  FOREIGN KEY (email_hote) REFERENCES utilisateur(email_utilisateur)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (id_zone) REFERENCES zone_geo(id_zone)
    ON UPDATE CASCADE ON DELETE CASCADE
);


CREATE TABLE equipement (
  id_equipement INT NOT NULL PRIMARY KEY,
  nom_equipement VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE proposer (
   id_logement  INT NOT NULL,
   id_equipement INT NOT NULL,
   PRIMARY KEY (id_logement, id_equipement),
   FOREIGN KEY (id_logement) REFERENCES logement(id_logement)
      ON UPDATE CASCADE ON DELETE CASCADE,
   FOREIGN KEY (id_equipement) REFERENCES equipement(id_equipement)
      ON UPDATE CASCADE ON DELETE CASCADE
);



CREATE TABLE moyen_paiement (
   id_moyen  INT  NOT NULL PRIMARY KEY,
   type_moyen_paiement VARCHAR(20)  NOT NULL,
   email_utilisateur   VARCHAR(100) NOT NULL,
   FOREIGN KEY (email_utilisateur) REFERENCES utilisateur(email_utilisateur)
      ON UPDATE CASCADE ON DELETE CASCADE
);



CREATE TABLE reservation (
   numero_reservation   INT   NOT NULL PRIMARY KEY,
   date_arrivee  DATE  NOT NULL,
   date_depart   DATE  NOT NULL,
   nb_voyageurs  INT  NOT NULL,
   statut_reservation   VARCHAR(15)  NOT NULL DEFAULT 'en_attente',
   montant_total  DECIMAL(9,2) NOT NULL,
   email_locataire  VARCHAR(100) NOT NULL,
```

```
  id_logement   INT    NOT NULL,
  FOREIGN KEY (email_locataire) REFERENCES utilisateur(email_utilisateur)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (id_logement) REFERENCES logement(id_logement)
    ON UPDATE CASCADE ON DELETE CASCADE
);



CREATE TABLE paiement (
  ref_transaction   VARCHAR(30)  NOT NULL PRIMARY KEY,
  montant_paiement   DECIMAL(9,2) NOT NULL,
  devise   CHAR(3)   NOT NULL DEFAULT 'EUR',
  date_paiement  DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  statut_paiement  VARCHAR(10)  NOT NULL,
  numero_reservation   INT  NOT NULL,
  id_moyen   INT  NOT NULL,
  ref_transaction_origine   VARCHAR(30),
  FOREIGN KEY (numero_reservation) REFERENCES reservation(numero_reservation)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (id_moyen) REFERENCES moyen_paiement(id_moyen)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (ref_transaction_origine) REFERENCES paiement(ref_transaction)
    ON UPDATE CASCADE ON DELETE CASCADE
);



CREATE TABLE avis (
  id_avis    INT  NOT NULL PRIMARY KEY,
```

```
  note_globale   INT  NOT NULL,
  commentaire_avis   VARCHAR(1000),
  numero_reservation INT   NOT NULL UNIQUE,
  FOREIGN KEY (numero_reservation) REFERENCES reservation(numero_reservation)
    ON UPDATE CASCADE ON DELETE CASCADE
);


CREATE TABLE recherche (
  id_recherche     INT   NOT NULL PRIMARY KEY,
  date_recherche    DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
  email_utilisateur VARCHAR(100) NOT NULL,
  id_zone  INT,
  FOREIGN KEY (email_utilisateur) REFERENCES utilisateur(email_utilisateur)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (id_zone) REFERENCES zone_geo(id_zone)
    ON UPDATE CASCADE ON DELETE SET NULL
);
```

CONTRAINTES

```
USE plateforme_location;


ALTER TABLE utilisateur ADD CONSTRAINT utilisateur_email
  CHECK (email_utilisateur LIKE '%_@_%.__%');
```

```
ALTER TABLE utilisateur ADD CONSTRAINT utilisateur_statut
  CHECK (statut_compte IN ('actif', 'suspendu'));

ALTER TABLE utilisateur ADD CONSTRAINT utilisateur_majeur
  CHECK (
    EXTRACT(YEAR FROM date_inscription) - EXTRACT(YEAR FROM date_naissance) > 18
    OR (EXTRACT(YEAR FROM date_inscription) - EXTRACT(YEAR FROM date_naissance) = 18
      AND EXTRACT(MONTH FROM date_inscription) * 100 + EXTRACT(DAY FROM date_inscription)
          >= EXTRACT(MONTH FROM date_naissance) * 100 + EXTRACT(DAY FROM date_naissance))
  );

ALTER TABLE utilisateur ADD CONSTRAINT utilisateur_iban
  CHECK (iban IS NULL OR (CHAR_LENGTH(iban) BETWEEN 15 AND 34 AND iban NOT LIKE '% %'));

ALTER TABLE utilisateur ADD CONSTRAINT utilisateur_iban_uniq UNIQUE (iban);

ALTER TABLE logement ADD CONSTRAINT logement_type
  CHECK (type_logement IN ('chambre', 'studio', 'appartement', 'loft', 'maison', 'villa'));
```

```
ALTER TABLE logement ADD CONSTRAINT logement_capacite
   CHECK (capacite_max BETWEEN 1 AND 20);


ALTER TABLE logement ADD CONSTRAINT logement_latitude
   CHECK (latitude BETWEEN -90 AND 90);
ALTER TABLE logement ADD CONSTRAINT logement_longitude
   CHECK (longitude BETWEEN -180 AND 180);


ALTER TABLE logement ADD CONSTRAINT logement_hote_titre UNIQUE (email_hote,
titre_logement);


ALTER TABLE photo ADD CONSTRAINT photo_numero CHECK (numero_photo >= 1);


ALTER TABLE moyen_paiement ADD CONSTRAINT moyen_type
   CHECK (type_moyen_paiement IN ('carte_bancaire', 'paypal', 'virement',
'apple_pay'));
```

```
ALTER TABLE reservation ADD CONSTRAINT reservation_dates
   CHECK (date_depart > date_arrivee);


ALTER TABLE reservation ADD CONSTRAINT reservation_voyageurs
   CHECK (nb_voyageurs >= 1);


ALTER TABLE reservation ADD CONSTRAINT reservation_statut
   CHECK (statut_reservation IN ('en_attente', 'confirmee', 'annulee', 'terminee'));


ALTER TABLE paiement ADD CONSTRAINT paiement_statut
   CHECK (statut_paiement IN ('reussi', 'echoue', 'rembourse'));


ALTER TABLE paiement ADD CONSTRAINT chk_paiement_devise
   CHECK (devise IN ('EUR', 'USD', 'GBP', 'CHF'));
```

```
ALTER TABLE avis ADD CONSTRAINT avis_note
    CHECK (note_globale BETWEEN 1 AND 5);
```

CONSIGNES

D'abord, génère les données en respectant toutes les contraintes ci-dessus (types, tailles, clés primaires, clés étrangères, CHECK, UNIQUE, valeurs par défaut). Fournis uniquement des requêtes INSERT INTO nom_table (colonnes) VALUES (...), avec plusieurs lignes par requête, sans explication et avec un commentaire -- avant chaque table. Insère dans l'ordre qui évite les erreurs de clé étrangère : zone_geo (pays, puis villes, puis quartiers, les parents avant les enfants), utilisateur, logement, equipement, proposer, photo, moyen_paiement, reservation, paiement, avis, recherche.

Ensuite, respecte les volumes suivants : 40 zones, 70 utilisateurs (dont 16 hôtes qui ont un IBAN et 4 comptes suspendus), 40 logements, 14 équipements avec 4 à 9 équipements par logement, 3 à 5 photos par logement, environ 400 réservations, 500 à 600 paiements, 150 à 200 avis et 160 recherches.

Enfin, rends les données réalistes et cohérentes avec le fonctionnement de l'entreprise : vraies villes et quartiers d'Europe, prénoms et noms, titres d'annonces et commentaires d'avis en français, coordonnées GPS proches du centre de la ville, prix adaptés au type de logement, . Un hôte ne réserve pas son propre logement. Deux réservations d'un même logement ne se chevauchent pas. Le nombre de voyageurs ne dépasse pas la capacité du logement. Les séjours déjà terminés ont le statut 'terminee' ou 'annulee', ceux à venir ont le statut 'confirmee', 'en_attente' ou 'annulee'. Un avis n'existe que pour une réservation 'terminee'. Le moyen de paiement d'un paiement appartient au locataire de la réservation. Un remboursement (statut 'rembourse') référence un paiement 'reussi' de la même réservation et son montant ne dépasse pas celui du paiement d'origine. Il doit y avoir au moins 3 logements jamais réservés et 8 utilisateurs actifs qui n'ont fait aucune réservation.